

How the repeatability pipeline works

The pipeline looks for video topics that have worked again and again for channels like yours. It finds videos that blew past their channel's normal views, boils them down to topics, then searches YouTube for each topic (and for the phrasings viewers really type) to count how many other channels also had a hit with it. A topic that many similar-sized channels have recently hit with is a repeatable idea.



One number drives everything: a channel's **median views** over its recent uploads. Every video is judged against its own channel's median, so a 20,000-view video on a small channel counts for more than a 200,000-view video on a huge one.

The pipeline, step by step

Before the steps: the median baseline

For your channel and every channel the pipeline touches, take the most recent uploads in the chosen format (long-form or Shorts) and find the median views. Your own median is used later, in step 3, to decide which channels are 'your size'.

```
channel_median = median( views of the last UPLOADS_FOR_MEDIAN uploads in FORMAT )
my_median      = channel_median( MY_CHANNEL )
```

Tuning the knobs

Knob	Turn it up	Turn it down
UPLOADS_FOR_MEDIAN now: 50 (YouTube caps this at the last 50 uploads)	A steadier median that one viral video can't move. But it looks further back, so a channel that grew recently gets a median that is too low and its normal videos look like outliers.	Reflects how the channel performs right now, but with few videos the median gets jumpy and outlier multiples become less trustworthy.
FORMAT now: "longform" (Shorts are 60s or less)	Not a dial. Long-form and Shorts get very different view counts, so they are never mixed. Set this to the format you plan to make; ideas proven in one format may not carry to the other.	

↓
a median for every channel

Step 1: Find outliers

Go through every upload on the channels you chose to scan. Keep the videos that did many times better than their channel's median, rank them by how far they beat it, and carry the best ones forward.

```
multiple = video_views ÷ channel_median
outlier  = multiple ≥ OUTLIER_MULTIPLE
keep the TOP_CANDIDATES outliers with the highest multiple
```

Tuning the knobs

KNOB	TURN IT UP	TURN IT DOWN
CHANNELS_TO_SCAN now: 23 channels	Add channels: a wider pool of topics. This list is the only source of ideas, so the final suggestions can never be more relevant to your niche than these channels are.	Remove channels: drop any that aren't really in your niche. One off-niche channel with a viral video can take a candidate slot and produce an idea you'd never make.
OUTLIER_MULTIPLE now: 5.0x	Only true breakouts survive. Fewer candidates, and more of them are one-offs (news, luck, a big collab) that are hard to repeat.	More candidates and more topic variety, but some are just a channel's normal good week rather than proof that the topic pulls views.
TOP_CANDIDATES now: 10 videos	The main lever for how many ideas you get. More videos in means more distinct topics out of step 2. Each extra topic costs one YouTube search (100 quota units) in step 3.	Fewer topics to build ideas from. 10 candidates can merge into as few as 3 topics, which leaves the final list leaning on query variants of the same few topics.

Watch out: candidates are ranked by multiple, not by views. Small channels have small medians, so one lucky video gives them a huge multiple and they can crowd out steadier channels.

↓
up to 10 outlier videos

Step 2: Merge into topics

Several outliers are often about the same thing. An LLM reads the titles, groups videos on the same underlying topic, and writes one short, generic YouTube search query per topic. Each query is then sent to YouTube's search autocomplete (free, no quota) to get the phrasings viewers really type, and the LLM picks the few worth scoring: same language, still on topic, a distinct angle. There is no math here, but these queries decide what step 3 measures.

```
N outlier videos → LLM → K topics    (K ≤ N)
each topic = name + search_query + the videos it came from

queries per topic = search_query + up to QUERY_VARIANTS autocomplete picks
```

Tuning the knobs

KNOB	TURN IT UP	TURN IT DOWN
LLM_PROVIDER / LLM_MODELS now: openai, gpt-4o-mini	Not a dial. A stronger model groups more sensibly and writes search queries closer to what viewers really type. A weak model tends to lump loosely related videos together, which means fewer and vaguer ideas.	
PROMPT (step2_topics.py) how broad topics and queries should be	Broader topics and queries ("AI course"): every search returns lots of hits, so scores look great, but the idea is too vague to act on and the hits are about many different things.	Narrower topics and queries ("Claude Code tutorial for beginners"): specific, usable ideas, but fewer search results qualify, so more topics fail MIN_HITS and get dropped.
QUERY_VARIANTS now: 3 per topic	More specific angles are tested per topic ("...for beginners", "...for business"), so the final list is more actionable and one broad topic can yield several ideas. Each variant costs one step 3 search (100 quota units).	Fewer angles. At 0 only the LLM's own, usually broad, query is scored: cheapest, but broad queries give vague ideas with inflated scores.

Answers are cached for a day, so the same candidates always give the same topics until the cache expires or you run with --refresh.



a few search queries per topic

Step 3: Score repeatability

Search YouTube for each query and look at the videos from the first channels that show up (yours is skipped). A result is a **hit** if it beat its own channel's median by enough, and is recent and watched enough for that comparison to mean something. Each hit gets a weight between 0 and 1: recent hits and hits from channels near your size count the most. The query's score is the sum of its hit weights. Every query is ranked as an idea of its own; one whose hits mostly repeat a better idea's hits is skipped as a duplicate. To qualify, an idea needs hits from enough different channels and a high enough score.

```
hit                = video_views ≥ HIT_MULTIPLE × channel_median
                   AND video_views ≥ MIN_HIT_VIEWS  AND age_days ≤ MAX_HIT_AGE_DAYS

recency            = 0.5 ^ ( age_days ÷ AGE_HALF_LIFE_DAYS )
size_gap           = | log10(channel_median) - log10(my_median) |
hit_weight         = recency ÷ ( 1 + size_gap )

repeatability      = sum of hit_weight over all of the query's hits
duplicate          = share of hits also in a better idea > MAX_HIT_OVERLAP
qualifies          = channels with a hit ≥ MIN_HITS  AND  repeatability ≥ MIN_SCORE
final list         = qualifying queries, not duplicates, top TOP_IDEAS
```

Reading the weight: a brand-new hit from a channel your size = 1.0. A channel 10x bigger or smaller halves it (0.5); 100x cuts it to a third. Every 180 days of age halves it again. So a one-year-old hit from a channel 100x your size adds only about 0.08.

Tuning the knobs: what counts as a hit

KNOB	TURN IT UP	TURN IT DOWN
HIT_MULTIPLE now: 4.0x	A hit has to be a real breakout. Fewer hits and lower scores overall, but each hit is strong evidence the topic itself pulled the views.	Mild over-performers count as hits. Scores rise across the board and the ranking gets noisier, because ordinary ups and downs are being counted as proof.
MIN_HIT_VIEWS now: 1000 views	Removes 'hits' that are tiny numbers on dead channels (17 views against a median of 3). Too high and you lose genuine hits from small channels, which are the ones most like yours.	Lets very small channels count. Because hits from channels near your size weigh the most, a small channel owner feels this noise more than anyone.
MAX_HIT_AGE_DAYS now: 730 days	Older videos can be hits. But an old video's views piled up over years and are compared with the channel's median today, so old videos pass too easily and evergreen topics look stronger than they are.	Only recent videos are evidence, answering 'could a new video on this break out now?'. Evergreen topics then need recent hits to qualify.
CHANNELS_PER_QUERY now: 20 channels	More channels checked per topic, so more chances to find hits and a fairer score. Costs a little more quota per topic.	Only the top-ranked channels are checked. Those are usually big channels, whose hits are weighted down anyway, so scores drop and fewer topics reach MIN_HITS.
SEARCH_RESULTS_PER_QUERY now: 50 (YouTube's maximum)	Already at the maximum.	A smaller sample of videos per topic: fewer hits found, lower scores, more topics dropped. It doesn't save quota (a search costs 100 units either way), so leave it at 50.

Tuning the knobs: scoring and qualifying

KNOB	TURN IT UP	TURN IT DOWN
AGE_HALF_LIFE_DAYS now: 180 days	Old hits keep their weight, which favours evergreen topics. The risk is recommending a topic whose moment has passed.	Only what is working right now scores well. Good for riding trends, but the trend may be over by the time your video is out.
MIN_HITS now: 6 channels	Only ideas proven on many different channels get through. Safer suggestions, but fewer of them, and newer or more niche topics get cut.	More ideas get through, including ones where just two or three channels got lucky. Riskier, but better at surfacing early trends.
MIN_SCORE now: 1.0	Demands more fresh, same-size evidence. 1.0 is worth one brand-new hit from a channel your size. Cuts ideas whose hits are many but stale or from far bigger channels.	Weakly supported ideas can fill leftover slots in the final list just because nothing better was found.
MAX_HIT_OVERLAP now: 0.5	Near-identical queries survive side by side, so the list can fill up with rewordings of one idea backed by the same videos.	Ideas must be backed by mostly different videos. A more varied list, but a genuinely distinct angle can be dropped because it shares hits with a broader, higher-scoring query.

Tuning the knobs: scoring and qualifying

KNOB

TURN IT UP

TURN IT DOWN

TOP_IDEAS
now: 5 ideas

Only a cap on how many ideas are printed. It never creates more ideas: if fewer topics pass MIN_HITS and MIN_SCORE, you get fewer. It has no effect on quality.

Two things to keep in mind. The score is a sum, so a broad query that returns many weak hits can outrank a focused one with a few strong hits: always look at the hit list, not just the score. And the size weighting is built into the formula (stats.hit_weight), not config.py; it is driven by your own median, so it shifts on its own as your channel grows.



ranked ideas, each with its hits as evidence

Top ideas: quick tuning guide

IF YOU WANT...	TRY THIS
More ideas in the final list	Raise TOP_CANDIDATES first, then QUERY_VARIANTS. If ideas are being dropped in step 3, lower MIN_HITS or MIN_SCORE.
Safer, more proven ideas	Raise MIN_HITS, MIN_SCORE and HIT_MULTIPLE.
Ideas that are working right now	Lower AGE_HALF_LIFE_DAYS (for example 90) and MAX_HIT_AGE_DAYS (for example 365).
Evergreen ideas	Raise AGE_HALF_LIFE_DAYS (for example 365) and keep MAX_HIT_AGE_DAYS generous (730 to 1095).
More specific, actionable ideas	Raise QUERY_VARIANTS, tighten the step 2 PROMPT towards narrower topics, or use a stronger LLM.
Ideas closer to your niche	Prune CHANNELS_TO_SCAN down to channels whose audience you actually want.